Tugas Pertemuan 1	
IF21W0508 – Praktikum Web	
NPM	243040054
Nama	Rifki Nurohmat Ramadani
Kelas	F
Link Github Repository	PW2026_243040054/pertemuan2/src/Tugas2 at main · rifkinrmdni/PW2026_243040054

Bagian 1. Screenshot Semua Kode

- Main.tsx



```tsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './Tugas2/App';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

- Types.ts



```ts
// src/Tugas2/types.ts
// Definisi tipe data yang dipakai bersama oleh beberapa komponen

export type JenisTransaksi = 'pemasukan' | 'pengeluaran';

export interface Transaksi {
  id: number;
  keterangan: string;
  nominal: number;
  jenis: JenisTransaksi;
}

// Data dari formulir: sama seperti Transaksi, tetapi belum punya id
export type DataTransaksiBaru = Omit<Transaksi, 'id'>;
```

- FormatRupiah.ts



```ts
// src/Tugas2/formatRupiah.ts
// Fungsi bantu: 5000 -> "Rp 5.000"
export function formatRupiah(angka: number): string {
  return 'Rp ' + angka.toLocaleString('id-ID');
}
```

- Kartu.tsx



```tsx
// src/Tugas2/Kartu.tsx
// Komponen pembungkus: judul lewat props, isi lewat children
import type { ReactNode } from 'react';

interface KartuProps {
  judul: string;
  children: ReactNode;
}

function Kartu({ judul, children }: KartuProps) {
  return (
    <section className="kartu">
      <h2>{judul}</h2>
      {children}
    </section>
  );
}

export default Kartu;
```

- FormatTransaksi.tsx

```tsx
// src/Tugas2/FormTransaksi.tsx
import { useState } from 'react';
import type { FormEvent } from 'react';
import type { DataTransaksiBaru, JenisTransaksi } from './types';

interface FormTransaksiProps {
  onTambah: (data: DataTransaksiBaru) => void;
}

function FormTransaksi({ onTambah }: FormTransaksiProps) {
  // Tiga state lokal: satu untuk setiap input (formulir terkendali)
  const [keterangan, setKeterangan] = useState<string>('');
  const [nominal, setNominal] = useState<string>('');
  const [jenis, setJenis] = useState<JenisTransaksi>('pemasukan');

  // Validasi: keterangan tidak kosong (maks. 40 karakter), nominal harus > 0
  const terlaluPanjang = keterangan.length > 40;
  const nominalAngka = Number(nominal); // input selalu teks -> ubah jadi angka
  const tidakValid =
    keterangan.trim() === '' ||
    terlaluPanjang ||
    nominal === '' ||
    nominalAngka <= 0;

  const tanganiKirim = (event: FormEvent<HTMLFormElement>) => {
    event.preventDefault(); // cegah halaman reload
    if (tidakValid) return;

    onTambah({
      keterangan: keterangan.trim(),
      nominal: nominalAngka,
      jenis: jenis,
    }); // "melapor" ke komponen induk

    setKeterangan('');
    setNominal('');
    setJenis('pemasukan');
  };

  return (
    <form onSubmit={tanganiKirim} className="form-transaksi">
      <input
        className="input"
        type="text"
        placeholder="Keterangan, contoh: Uang saku"
        value={keterangan}
        onChange={(event) => setKeterangan(event.target.value)}
      />
      {terlaluPanjang && (
        <p className="galat">Keterangan maksimal 40 karakter.</p>
      )}

      <div className="baris">
        <input
          className="input"
          type="number"
          placeholder="Nominal (Rp)"
          min="1"
          value={nominal}
          onChange={(event) => setNominal(event.target.value)}
        />
        <select
          className="input"
          value={jenis}
          onChange={(event) => setJenis(event.target.value as JenisTransaksi)}
        >
          <option value="pemasukan">Pemasukan</option>
          <option value="pengeluaran">Pengeluaran</option>
        </select>
      </div>

      <button className="tombol" type="submit" disabled={tidakValid}>
        Tambah Transaksi
      </button>
    </form>
  );
}

export default FormTransaksi;
```

- ItemTransaksi.tsx



```tsx
// src/Tugas2/ItemTransaksi.tsx
import { formatRupiah } from "./FormatRupiah";
import type { Transaksi } from "./types";

interface ItemTransaksiProps {
  transaksi: Transaksi;
  onHapus: (id: number) => void;
}

function ItemTransaksi({ transaksi, onHapus }: ItemTransaksiProps) {
  const pemasukan = transaksi.jenis === "pemasukan";

  return (
    <li className="item">
      <span>{transaksi.keterangan}</span>
      {/* className dinamis: hijau untuk pemasukan, merah untuk pengeluaran */}
      <strong className={pemasukan ? "hijau" : "merah"}>
        {pemasukan ? "+" : "-"} {formatRupiah(transaksi.nominal)}
      </strong>
      <button
        className="tombol tombol-merah tombol-kecil"
        onClick={() => onHapus(transaksi.id)}
      >
        Hapus
      </button>
    </li>
  );
}

export default ItemTransaksi;
```

- RingkasanSaldo



```tsx
// src/Tugas2/RingkasanSaldo.tsx
import { formatRupiah } from "./FormatRupiah";

interface RingkasanSaldoProps {
  pemasukan: number;
  pengeluaran: number;
  saldo: number;
}

function RingkasanSaldo({
  pemasukan,
  pengeluaran,
  saldo,
}: RingkasanSaldoProps) {
  return (
    <div>
      <p className="saldo-label">Total saldo</p>
      <p className={saldo < 0 ? "saldo merah" : "saldo"}>
        {saldo < 0 ? "-" : ""} {formatRupiah(Math.abs(saldo))}
      </p>
      <p className="ringkasan">
        Pemasukan: <span className="hijau">{formatRupiah(pemasukan)}</span> |
        Pengeluaran: <span className="merah">{formatRupiah(pengeluaran)}</span>
      </p>
    </div>
  );
}

export default RingkasanSaldo;
```

- App.tsx

```tsx
// src/Tugas2/App.tsx
import { useState, useEffect } from "react";
import "./App.css";
import Kartu from "./Kartu";
import FormTransaksi from "./FormTransaksi";
import ItemTransaksi from "./ItemTransaksi";
import RingkasanSaldo from "./RingkasanSaldo";
import type { Transaksi, DataTransaksiBaru } from "./types";

function App() {
  // 1. State utama: baca dari localStorage dulu (hanya dijalankan sekali)
  const [daftarTransaksi, setDaftarTransaksi] = useState<Transaksi[]>(() => {
    const dataTersimpan = localStorage.getItem("catatan-keuangan");
    return dataTersimpan ? (JSON.parse(dataTersimpan) as Transaksi[]) : [];
  });

  // 2. Efek: simpan ke localStorage setiap daftarTransaksi berubah
  useEffect(() => {
    localStorage.setItem("catatan-keuangan", JSON.stringify(daftarTransaksi));
  }, [daftarTransaksi]);

  // 3. Menambah transaksi: array BARU (spread), tidak memakai push
  const tambahTransaksi = (data: DataTransaksiBaru) => {
    const transaksiBaru: Transaksi = { id: Date.now(), ...data };
    setDaftarTransaksi([...daftarTransaksi, transaksiBaru]);
  };

  // 4. Menghapus transaksi: .filter() menghasilkan array baru
  const hapusTransaksi = (id: number) => {
    setDaftarTransaksi(daftarTransaksi.filter((t) => t.id !== id));
  };

  // 5. Nilai turunan: dihitung dari state, TIDAK disimpan sebagai state baru
  const totalPemasukan = daftarTransaksi
    .filter((t) => t.jenis === "pemasukan")
    .reduce((jumlah, t) => jumlah + Number(t.nominal), 0);
  const totalPengeluaran = daftarTransaksi
    .filter((t) => t.jenis === "pengeluaran")
    .reduce((jumlah, t) => jumlah + Number(t.nominal), 0);
  const saldo = totalPemasukan - totalPengeluaran;

  return (
    <div className="wadah">
      <h1 className="judul">Catatan Keuangan Mini</h1>
      <p className="subjudul">Rifki Nurohmat Ramadani_243040054</p>

      <Kartu judul="Saldo">
        <RingkasanSaldo
          pemasukan={totalPemasukan}
          pengeluaran={totalPengeluaran}
          saldo={saldo}
        />
      </Kartu>

      <Kartu judul="Tambah Transaksi">
        <FormTransaksi onTambah={tambahTransaksi} />
      </Kartu>

      <Kartu judul="Daftar Transaksi">
        {/* Conditional rendering: pesan khusus jika belum ada transaksi */}
        {daftarTransaksi.length === 0 ? (
          <p className="kosong">
            Belum ada transaksi. Catat pemasukan atau pengeluaran pertamamu!
          </p>
        ) : (
          <ul className="daftar">
            {daftarTransaksi.map((t) => (
              <ItemTransaksi
                key={t.id}
                transaksi={t}
                onHapus={hapusTransaksi}
              />
            ))}
          </ul>
        )}
      </Kartu>
    </div>
  );
}

export default App;
```

- App.css



```css
/* src/Tugas2/App.css */
.wadah {
  max-width: 640px;
  margin: 40px auto;
  padding: 0 20px;
  font-family: Arial, sans-serif;
  color: #1f2937;
  text-align: center;
}
.judul {
  color: #1b365d;
  margin-bottom: 4px;
  font-size: 28px;
  line-height: 1.3;
}
.subjudul {
  color: #6b7280;
  margin-top: 0;
  margin-bottom: 24px;
}

.kartu {
  background: #ffffff;
  border: 1px solid #d0d7de;
  border-radius: 10px;
  padding: 20px;
  margin-bottom: 20px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.08);
}
.kartu h2 {
  margin-top: 0;
  font-size: 18px;
  color: #1b365d;
}

.baris {
  display: flex;
  gap: 8px;
}
.form-transaksi {
  display: flex;
  flex-direction: column;
  gap: 10px;
}
.input {
  flex: 1;
  padding: 10px;
  border: 1px solid #d0d7de;
  border-radius: 6px;
  font-size: 14px;
}

.tombol {
  padding: 10px 16px;
  border: none;
  border-radius: 6px;
  background: #1b365d;
  color: #ffffff;
  font-weight: bold;
  cursor: pointer;
}
.tombol:disabled {
  background: #9ca3af;
  cursor: not-allowed;
}
.tombol-merah {
  background: #dc2626;
}
.tombol-kecil {
  padding: 6px 10px;
  font-size: 12px;
}

.daftar {
  list-style: none;
  padding: 0;
  margin: 0;
}
.item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 10px 0;
  border-bottom: 1px solid #e5e7eb;
}
.item span {
  flex: 1;
}

.hijau {
  color: #16a34a;
}
.merah {
  color: #dc2626;
}

.saldo-label {
  margin: 0;
  color: #6b7280;
  font-size: 13px;
}
.saldo {
  margin: 4px 0 12px;
  font-size: 32px;
  font-weight: bold;
  color: #1b365d;
}
.saldo.merah {
  color: #dc2626;
}

.kosong {
  color: #6b7280;
  text-align: center;
  font-style: italic;
}
.galat {
  color: #dc2626;
  font-size: 13px;
  margin: 0;
}
.ringkasan {
  font-size: 14px;
  color: #374151;
  margin: 0;
}
```

Bagian 2. Penjelasan Kode

main.tsx. Titik masuk aplikasi. Berkas ini mengimpor komponen App dari folder Tugas2 lalu menampilkannya ke elemen root di index.html. StrictMode membantu menemukan bug saat pengembangan.

types.ts. Berisi definisi tipe data. Transaksi memuat id, keterangan, nominal, dan jenis. JenisTransaksi hanya boleh bernilai 'pemasukan' atau 'pengeluaran', sehingga salah ketik akan terdeteksi TypeScript. DataTransaksiBaru dibuat dengan Omit<Transaksi, 'id'> karena data dari formulir belum punya id; id dibuat di App.

FormatRupiah.ts. Fungsi bantu yang mengubah angka menjadi format rupiah, misalnya 5000 menjadi Rp 5.000, dengan toLocaleString('id-ID'). Fungsi ini dipisah karena dipakai di dua komponen (ItemTransaksi dan RingkasanSaldo), sehingga kode tidak ditulis berulang.

Kartu.tsx. Komponen pembungkus yang menerima judul dan children. Isi apa pun di antara <Kartu> dan </Kartu> akan ditampilkan di dalam kartu. Komponen ini dipakai tiga kali di App (Saldo, Tambah Transaksi, Daftar Transaksi), sehingga gaya kartu konsisten dan kodenya efisien.

FormTransaksi.tsx. Formulir terkendali dengan tiga state lokal: keterangan, nominal, dan jenis. Setiap input memakai value dan onChange, sehingga isinya selalu sama dengan state. Validasi memastikan keterangan tidak kosong dan tidak lebih dari 40 karakter, serta nominal lebih dari 0; jika tidak valid, tombol dinonaktifkan. event.preventDefault() mencegah halaman reload. Nominal diubah dengan Number(nominal) karena nilai input selalu berupa teks. Setelah valid, komponen memanggil onTambah untuk melapor ke induk, lalu mengosongkan input.

ItemTransaksi.tsx. Menampilkan satu transaksi. Komponen ini menerima objek transaksi dan fungsi onHapus lewat props. className dinamis memberi warna hijau

untuk pemasukan dan merah untuk pengeluaran. Tombol hapus memakai onClick={() => onHapus(transaksi.id)}; arrow function dipakai karena kita perlu mengirim id, dan tanpa itu fungsi akan langsung berjalan saat render.

RingkasanSaldo.tsx. Menampilkan total saldo, total pemasukan, dan total pengeluaran yang diterima lewat props. Jika saldo negatif, tampilannya berwarna merah. Komponen ini hanya menampilkan data dan tidak menyimpan state.

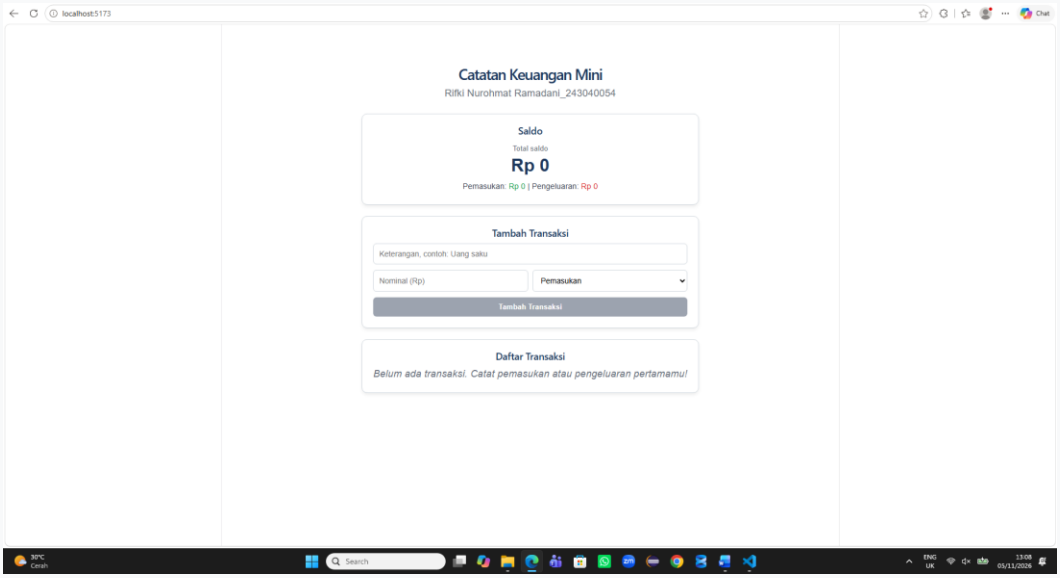
App.tsx. Komponen induk yang menyimpan state daftarTransaksi. Hal-hal pentingnya:

- State awal dibaca dari localStorage dengan lazy initializer useState(() => ...), sehingga hanya dijalankan sekali saat aplikasi pertama kali tampil.
- useEffect dengan dependency [daftarTransaksi] menyimpan data ke localStorage setiap data berubah, memakai JSON.stringify karena localStorage hanya menyimpan teks.
- tambahTransaksi memakai spread [...daftarTransaksi, transaksiBaru], dan hapusTransaksi memakai .filter(). Keduanya membuat array baru, tidak mengubah array lama, karena React mendeteksi perubahan dengan membandingkan data lama dan baru.
- Total pemasukan, pengeluaran, dan saldo dihitung dengan .filter() dan .reduce() langsung dari state, tidak dijadikan state baru, supaya tidak ada data ganda yang bisa tidak sinkron.
- Conditional rendering dengan ternary menampilkan pesan khusus jika daftar kosong.

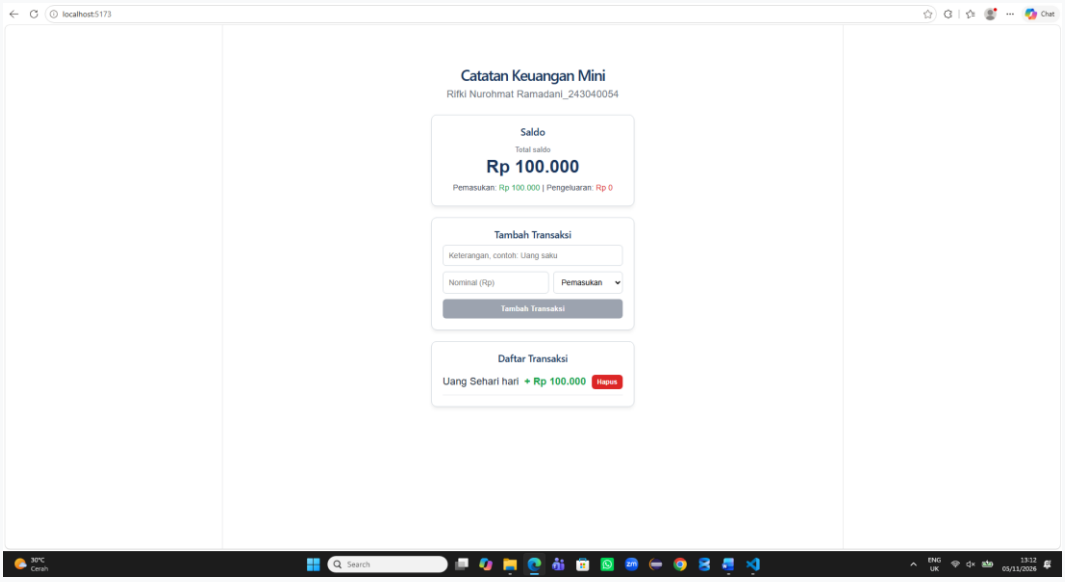
App.css. Berisi gaya tampilan. Kelas .hijau dan .merah mewarnai nominal, .kosong menata pesan daftar kosong, .galat menata pesan kesalahan, dan .tombol:disabled memberi tampilan abu-abu pada tombol yang nonaktif.

Bagian 3. Screenshot Hasil Antarmuka

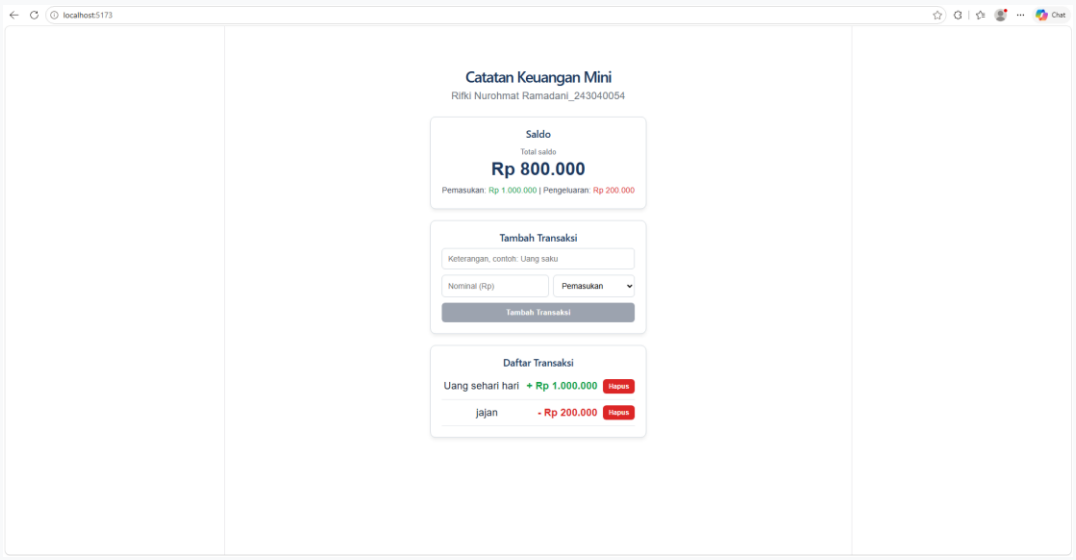
- Tampilan awal aplikasi saat belum ada transaksi



- Tampilan setelah menambah transaksi pemasukan



- Tampilan setelah menambah transaksi pengeluaran dan saldo terhitung otomatis



- Validasi formulir: tombol Tambah nonaktif saat keterangan kosong atau nominal 0



Bagian 4. Penjelasan Hasil Antarmuka

1. **Kondisi awal.** Daftar kosong sehingga muncul pesan khusus lewat conditional rendering. Saldo Rp 0 karena belum ada data untuk dijumlahkan, dan tombol nonaktif karena formulir belum valid.
2. **Pemasukan.** Transaksi tampil hijau dengan tanda “+”. Saldo bertambah otomatis karena dihitung ulang dari state setiap komponen dirender ulang.

3. **Pengeluaran.** Transaksi tampil merah dengan tanda “-” dan saldo berkurang. Ini terjadi karena setDaftarTransaksi membuat array baru, sehingga React merender ulang tampilan.
4. **Validasi kosong atau nol.** Tombol nonaktif karena nilai tidakValid bernilai true. Ini hanya mungkin karena input terkendali, jadi data input langsung bisa dipakai untuk validasi.
5. **Keterangan terlalu panjang.** Pesan galat muncul dari kondisi terlaluPanjang && ..., dan tombol tetap nonaktif sampai teks dipersingkat.
6. **Hapus transaksi.** Item hilang dan saldo ikut berubah. .filter() membuat array baru tanpa item tersebut, tanpa memutasi array lama.
7. **Refresh.** Data tetap ada karena useEffect sudah menyimpannya ke localStorage, dan saat aplikasi dimuat ulang, state awal dibaca dari sana. Tanpa useEffect, data akan kembali kosong karena state hanya hidup di memori.
8. **Local Storage.** Kunci catatan-keuangan berisi data dalam format JSON. JSON.stringify diperlukan karena localStorage hanya bisa menyimpan teks.
9. **Semua dihapus.** Pesan kosong muncul lagi dan data di localStorage ikut menjadi []. Ini menunjukkan tampilan dan penyimpanan selalu sinkron lewat useEffect.